

Learnlytics

.handbook

== | **MLOps &**
Model Deployment ==



Concise



Comprehensive



Career-Defining



LEARN SMART
ANALYZE BETTER.
GROW FASTER.



MLOps & Model Deployment

PART 01

A Complete 14-Chapter Study Guide

First Edition

Published by Learnlytics.handbook

www.learnlyticshandbook.shop | learnlytic.handbook@gmail.com

Copyright © 2026 Learnlytics.handbook. All rights reserved.

No part of this publication may be reproduced, distributed, stored in a retrieval system, or transmitted in any form or by any means — electronic, mechanical, photocopying, recording, or otherwise — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

For permission requests or licensing inquiries, write to the publisher at the address above.

ISBN: 978-1-2565-5895-9 (Digital Edition)

First published: 2026

Printed and distributed digitally worldwide.

***IMPORTANT: LICENSE & COPYRIGHT NOTICE** This document and all included materials are the exclusive intellectual property of Learnlytics.handbook. Your purchase grants you a single-user, personal license. This file is digitally tracked. Unauthorized reproduction, forwarding, resale, or distribution of this material — whether in private groups, Telegram channels, or public forums — is strictly prohibited and constitutes copyright infringement. Our legal team actively monitors digital distribution networks. Violations of this agreement will result in immediate termination of your access, notification to your educational/professional institution, and formal legal action under applicable international and domestic copyright laws. Thank you for respecting the hard work that went into creating this guide!*

Disclaimer: The information in this guide is provided for educational purposes. MLOps tools, cloud platforms, and deployment patterns described here reflect widely accepted industry standards. Always refer to official documentation for the latest API changes and platform-specific configurations. The author and publisher make no warranty, express or implied, with respect to the material contained herein.

Typeset in Arial | Cover design: Learnlytics.handbook

IN Made in India with ❤️ for ML Engineers Worldwide

<https://learnlyticshandbook.shop/>

@learnlytics.handbook

TABLE OF CONTENTS

14 Chapters • 112 Topics • A Complete MLOps & Deployment Journey

PART 1: From Experiment to Deployment

CHAPTER TITLE	PAGE
1 Chapter 1: MLflow — Experiment Tracking	08-35
1.1 What is MLflow and Why Experiment Tracking Matters 1.2 MLflow Architecture	
1.3 Setting Up MLflow 1.4 Logging Parameters	
1.5 Logging Metrics 1.6 Logging Artifacts	
1.7 Auto-Logging with mlflow.autolog 1.8 Comparing Runs in the MLflow UI	
2 Chapter 2: MLflow Model Registry	36-65
2.1 What is the Model Registry 2.2 Registering a Model	
2.3 Model Stages 2.4 Transitioning Model Stages	
2.5 Loading a Model from the Registry 2.6 Model Flavours	
2.7 Model Versioning Workflow 2.8 Integrating Registry with Deployment	
3 Chapter 3: DVC — Data Version Control	66-97
3.1 What is DVC and Why Data Versioning Matters 3.2 DVC vs. Git	
3.3 Setting Up DVC 3.4 Tracking Data Files	
3.5 DVC Remote Storage 3.6 DVC Pipelines	
3.7 DVC Experiments 3.8 Practical DVC Workflow	
4 Chapter 4: Weights & Biases (wandb)	98-130
4.1 What is Weights & Biases 4.2 Setting Up wandb	
4.3 Initialising a Run 4.4 Logging Metrics and Media	
4.5 Logging Model Artifacts 4.6 Sweeps	
4.7 wandb Dashboard 4.8 MLflow vs. wandb	
5 Chapter 5: FastAPI — Model Serving	131-165
5.1 What is Model Serving and Why FastAPI 5.2 FastAPI Basics	
5.3 Loading a Model at Startup 5.4 Building a Prediction Endpoint	
5.5 Request and Response Schemas 5.6 Running and Testing the API	
5.7 Error Handling in FastAPI 5.8 Deploying FastAPI with Docker	
6 Chapter 6: Streamlit — Model Demo Apps	166-197
6.1 What is Streamlit in MLOps Context 6.2 Streamlit Basics for MLOps	
6.3 Loading and Caching a Model 6.4 Building a Prediction Interface	
6.5 Displaying Model Outputs 6.6 Adding Visualisations	

[6.7](#) Deploying Streamlit to Streamlit Cloud [6.8](#) FastAPI vs. Streamlit

7 Chapter 7: Gradio **198-232**

[7.1](#) What is Gradio and How It Differs from Streamlit [7.2](#) Building a Basic Gradio Interface

[7.3](#) Input and Output Components [7.4](#) Gradio Blocks API

[7.5](#) Sharing Demos [7.6](#) Gradio for NLP and CV Models

[7.7](#) Adding Examples and Descriptions [7.8](#) Gradio vs. Streamlit for Model Demos

8 Chapter 8: BentoML **233-263**

[8.1](#) What is BentoML [8.2](#) Saving a Model with BentoML

[8.3](#) Creating a Service [8.4](#) Running a BentoML Service Locally

[8.5](#) Building a Bento [8.6](#) Containerising with BentoML

[8.7](#) Deploying a Bento [8.8](#) BentoML vs. FastAPI

9 Chapter 9: ONNX — Model Portability **264-294**

[9.1](#) What is ONNX [9.2](#) Why ONNX Matters

[9.3](#) Exporting a sklearn Model to ONNX [9.4](#) Exporting a PyTorch Model to ONNX

[9.5](#) Running Inference with ONNX Runtime [9.6](#) Validating ONNX Export

[9.7](#) ONNX Model Optimisation [9.8](#) When to Use ONNX

10 Chapter 10: Docker — Containerising Models **295-327**

[10.1](#) What is Docker and Why ML Models Need Containers [10.2](#) Docker Core Concepts

[10.3](#) Writing a Dockerfile for an ML Model [10.4](#) Building and Running a Docker Image

[10.5](#) Docker Compose for ML Services [10.6](#) Reducing Docker Image Size

[10.7](#) Pushing to Docker Hub or Container Registry [10.8](#) Docker Workflow for ML Projects

11 Chapter 11: Cloud Run / App Engine **328-355**

[11.1](#) Serverless vs. Always-On Deployment [11.2](#) What is Google Cloud Run

[11.3](#) Deploying a FastAPI Container to Cloud Run [11.4](#) Environment Variables and Secrets

[11.5](#) Cloud Run Scaling Configuration [11.6](#) What is App Engine

[11.7](#) Cloud Run vs. App Engine [11.8](#) Monitoring Cloud Run Deployments

12 Chapter 12: AWS SageMaker Basics **356-391**

[12.1](#) What is AWS SageMaker [12.2](#) SageMaker Notebooks

[12.3](#) SageMaker Training Jobs [12.4](#) SageMaker Model Registry

[12.5](#) SageMaker Endpoints [12.6](#) SageMaker Batch Transform

[12.7](#) SageMaker Pipelines [12.8](#) SageMaker Cost Management

13 Chapter 13: Google Vertex AI Basics **392-424**

[13.1](#) What is Google Vertex AI [13.2](#) Vertex AI Workbench

[13.3](#) Vertex AI Training [13.4](#) Vertex AI Model Registry

[13.5](#) Vertex AI Endpoints [13.6](#) Vertex AI Batch Prediction

[13.7](#) Vertex AI Pipelines [13.8](#) SageMaker vs. Vertex AI

- 14.1** What is CI/CD in Machine Learning **14.2** GitHub Actions Basics
14.3 Automated Testing for ML Code **14.4** Model Training in CI
14.5 Model Evaluation Gate **14.6** Automated Docker Build and Push
14.7 Continuous Deployment to Cloud Run **14.8** Secrets Management in GitHub Actions

MLOps & Model Deployment

Part 1: From Experiment to Deployment

Chapter 1: MLflow — Experiment Tracking

Target Audience: Junior to Mid-Level Data Scientists

Prerequisites: Basic Python & Docker Knowledge

Chapter 1: MLflow — Experiment Tracking

1.1 What is MLflow and Why Experiment Tracking Matters

► Definition

MLflow is an open-source platform created by Databricks that helps data scientists manage the entire machine learning lifecycle. Think of it as a 'lab notebook' for your ML experiments — it automatically records every experiment you run, so you never lose track of what you tried, what settings you used, or how well your model performed.

Experiment tracking specifically means recording every training run with its exact configuration (hyperparameters), the metrics it achieved (accuracy, loss, F1 score), and the files it produced (model weights, plots, data files). Without tracking, every run is a black box that you may never be able to reproduce.

► Why It Matters in MLOps

Without experiment tracking, production ML breaks in the following ways:

- You cannot reproduce your best model. After 50 runs, you have no record of which learning rate or tree depth gave you 94% accuracy — you just know 'it worked once'.
- Model debugging becomes guesswork. When a deployed model degrades, you cannot compare its training configuration against older, better-performing versions.
- Team collaboration collapses. Two data scientists working on the same problem silently overwrite each other's best results with no audit trail.
- Compliance fails. Regulated industries (finance, healthcare) require proof of exactly how a model was built. Without logs, you cannot provide this.
- Deployment becomes risky. DevOps teams cannot confidently deploy 'the best model' if there is no systematic record of which artifact that actually is.

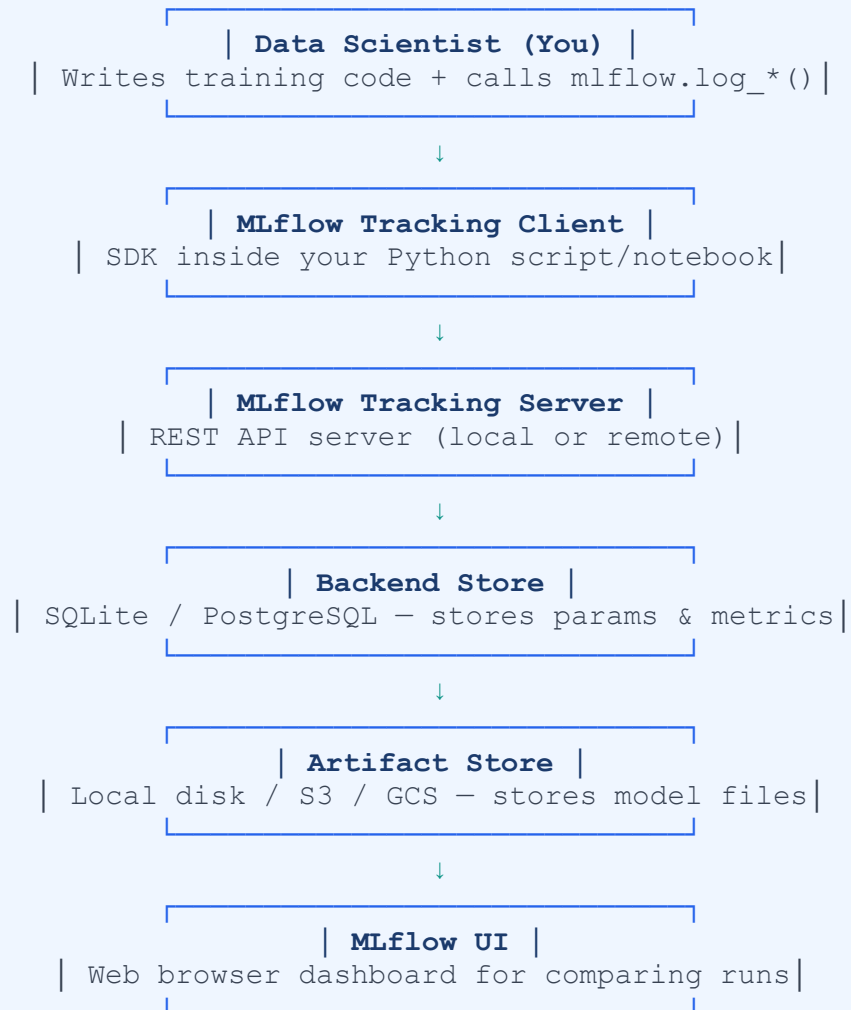
► Tool Overview

Question	Answer
What does it do?	Records parameters, metrics, code version, and output files for every ML training run automatically.
Who made it?	Databricks (open-source under Apache 2.0 licence); widely adopted across the industry since 2018.

What problem does it solve?	The 'I forgot which settings gave me the best result' problem — the most common pain point for data scientists.
How does it fit in MLOps?	It sits at the very start of the MLOps pipeline — the experimentation phase — before any model reaches staging or production.

► System Architecture

Here is how MLflow fits into the broader MLOps system:



► Step-by-Step Workflow

Here is the complete flow from experiment to insight:

1. You write a training script (e.g., `train.py`) with your model code.
2. You add `mlflow.start_run()` to start recording a new experiment run.

3. Inside the run, you call `mlflow.log_param()` for each hyperparameter you set.
4. After training, you call `mlflow.log_metric()` for each evaluation score.
5. You call `mlflow.log_artifact()` to save your model file or any output plots.
6. The MLflow Tracking Client sends all this data to the Tracking Server.
7. The server stores metadata (params/metrics) in the Backend Store and files in the Artifact Store.
8. You open the MLflow UI in your browser and compare all runs side by side.
9. You identify the best run and register that model version in the Model Registry.

► Industry Use Case

Real-World Example: Credit Scoring Model at a Bank

A team of 5 data scientists is building a credit risk model. Over 3 weeks they run 200+ experiments varying: number of trees (50–500), max depth (3–10), learning rate (0.01–0.3), and feature sets. Without MLflow, at week 3, no one remembers which combination gave the best AUC-ROC. With MLflow, every run is logged. The team filters the UI by AUC-ROC > 0.91, sorts by training time, and in 2 minutes identifies the production-ready model. The compliance team exports the run log as evidence of model validation. Time saved: ~2 days of re-running experiments.

1.2 MLflow Architecture

► Definition

MLflow architecture refers to the set of components that work together to capture, store, and display your experiment data. Understanding the architecture helps you decide where to run each component — on your laptop, on a team server, or in the cloud — and how the pieces communicate with each other.

► Why It Matters in MLOps

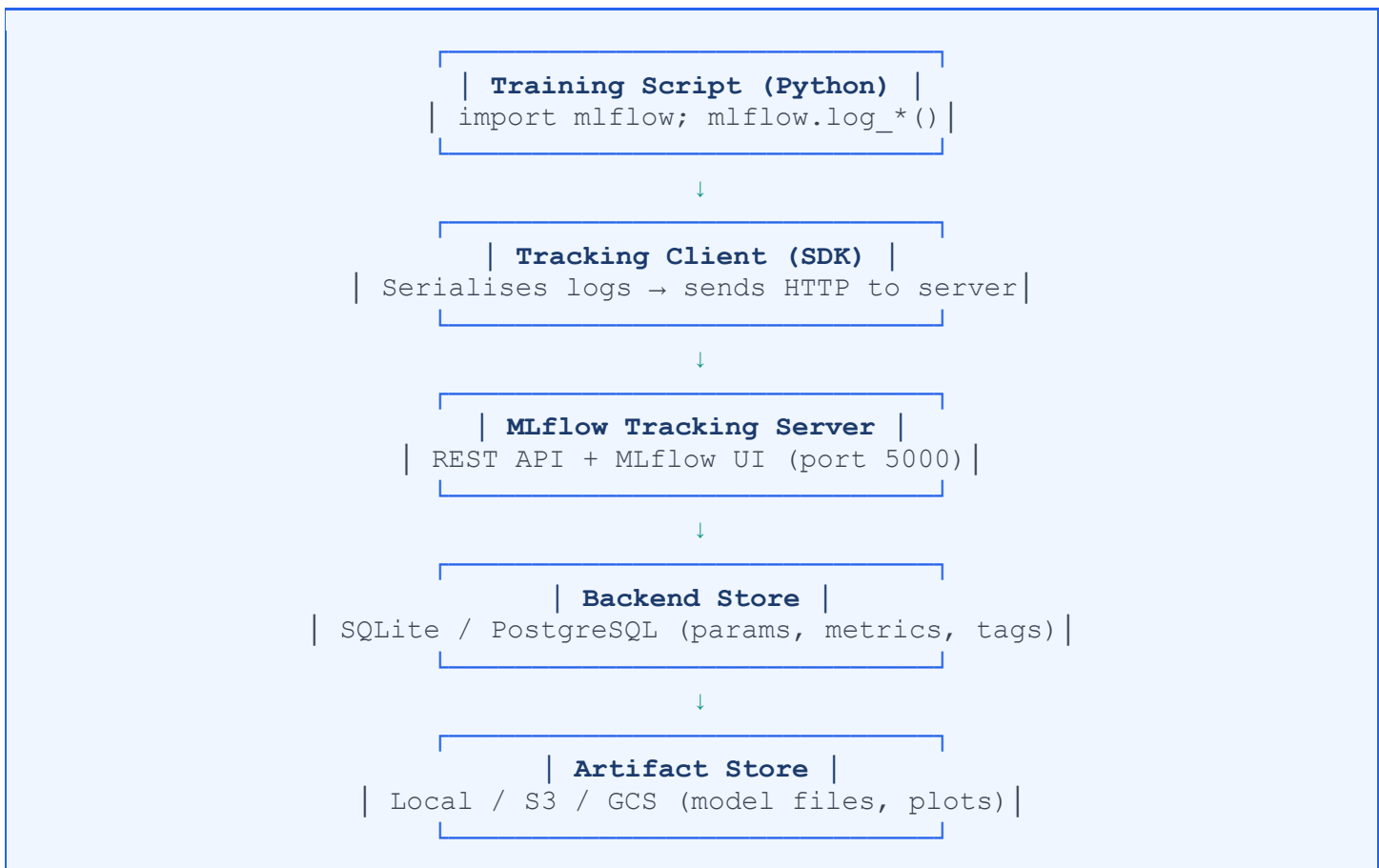
If you do not understand the architecture, the following problems occur:

- You accidentally run a local-only MLflow setup in production, meaning experiment data disappears when your machine restarts.
- Two team members cannot see each other's experiments because they each have a separate local tracking server.
- Model artifacts get saved to the wrong location, making deployment pipelines unable to find them.
- Scaling to cloud storage becomes difficult without knowing which component handles files vs. metadata.

► Tool Overview — Four Core Components

Component	What It Does
Tracking Server	A REST API server that receives logs from your training scripts. Can run locally (mlflow server) or on a remote machine.
Backend Store	A database that stores structured data — experiment names, run IDs, parameters, metrics, and tags. Supports SQLite (dev), PostgreSQL, MySQL (prod).
Artifact Store	File storage for large binary files — model weights (.pkl), plots (.png), datasets. Supports local disk, AWS S3, Google Cloud Storage, Azure Blob.
MLflow UI	A web dashboard served by the Tracking Server on port 5000 by default. Lets you search, filter, and compare runs visually.

► System Architecture



► Step-by-Step Workflow — How Data Flows

10. Your Python script calls `mlflow.log_param('lr', 0.01)`. The Tracking Client serialises this into a JSON payload.
11. The client sends an HTTP POST request to the Tracking Server's REST API (e.g., `http://localhost:5000/api/2.0/mlflow/runs/log-parameter`).
12. The Tracking Server validates the request, then writes the parameter name and value to the Backend Store (the database).
13. When you call `mlflow.log_artifact('model.pkl')`, the client uploads the file directly to the Artifact Store (bypassing the server for large files).
14. The server records the artifact path (pointer) in the Backend Store, linking it to the run.
15. When you open the MLflow UI, it queries the Backend Store to display run tables, charts, and provides download links to the Artifact Store.

► Local vs. Remote Architecture

Setup	When to Use
Local (default)	Individual development. <code>mlflow.set_tracking_uri('mlruns')</code> — all data goes to a local folder.
Local Server	Small team sharing one server. Run mlflow server on a shared machine; all teammates point to it.
Remote Server + Cloud Storage	Production / large teams. Server on EC2/GKE; Backend on RDS PostgreSQL; Artifacts on S3.
Managed (Databricks)	Enterprise teams. Fully managed MLflow with no infrastructure to maintain.

► Code & Commands — Checking Architecture in Code

```
import mlflow

# Check where MLflow is currently pointing
print(mlflow.get_tracking_uri())
# Default output: 'mlruns' (local folder)

# Point to a remote tracking server
mlflow.set_tracking_uri('http://my-mlflow-server:5000')

# Point to a local SQLite database as backend
mlflow.set_tracking_uri('sqlite:///mlflow.db')

# Set artifact location (usually done at server start, not in code)
# mlflow server --artifact-root s3://my-bucket/mlflow-artifacts
```

► Integration Map

MLflow Component	Integrates With
Tracking Client	scikit-learn, PyTorch, TensorFlow, XGBoost, LightGBM, HuggingFace Transformers
Backend Store	SQLite, PostgreSQL, MySQL, MSSQL, Amazon RDS, Google Cloud SQL
Artifact Store	Local filesystem, AWS S3, Azure Blob Storage, Google Cloud Storage, SFTP, HDFS
MLflow UI	Any web browser; can be reverse-proxied through NGINX
Tracking Server	Docker, Kubernetes, AWS EC2, Databricks, Azure ML, Google Vertex AI

► Industry Use Case

🚀 Startup Scaling from Laptop to Cloud

An ML startup begins with 2 data scientists running MLflow locally (sqlite backend + local artifacts). As the team grows to 8 scientists, they move the tracking server to an EC2 instance with a PostgreSQL RDS backend and S3 artifact store. All 8 scientists simply change their `mlflow.set_tracking_uri()` to point to the server — the rest of their code stays identical. MLflow's modular architecture makes this migration seamless in under a day.

1.3 Setting Up MLflow

► Definition

Setting up MLflow means installing the MLflow library in your Python environment, (optionally) starting a tracking server, and configuring your scripts to know where to send experiment data. It is the foundational step before you can log any experiments.

► Why It Matters in MLOps

- Without a proper setup, every experiment goes into a hidden local folder (`mlruns/`) that is not shared, not versioned, and often accidentally deleted.
- Teams without a central server end up with 8 separate local databases — no one can see anyone else's results.
- Deployment pipelines break because they cannot find model artifacts if the artifact store was not configured consistently.

► Setup & Configuration — Full Installation Guide

Step 1: Install MLflow

```
# Basic installation
pip install mlflow

# With extras for specific backends (choose as needed)
pip install mlflow[extras] # all extras
pip install psycopg2-binary # for PostgreSQL backend
pip install boto3 # for AWS S3 artifact store
pip install google-cloud-storage # for GCS artifact store

# Verify installation
mlflow --version
# Expected output: mlflow, version 2.x.x
```

Step 2: Start the Tracking Server (Recommended for Teams)

```
# Option A: Minimal local server (SQLite backend, local artifacts)
mlflow server \
  --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./mlartifacts \
  --host 0.0.0.0 \
  --port 5000

# Option B: Production server (PostgreSQL + S3)
mlflow server \
  --backend-store-uri postgresql://user:pass@db-host:5432/mlflow \
  --default-artifact-root s3://my-company-mlflow/artifacts \
  --host 0.0.0.0 \
  --port 5000

# Open the UI in your browser:
# http://localhost:5000
```

Step 3: Configure Your Script

```
import mlflow

# --- Configuration (do this once at the top of your script) ---
```

```
# Tell MLflow where your tracking server is
mlflow.set_tracking_uri('http://localhost:5000')

# Create or select an experiment (like a project folder)
# All runs for this project go here
mlflow.set_experiment('credit-risk-model')

# Now you're ready to log experiments!
```

► Common Setup Configurations

Scenario	Configuration Command
Solo dev on laptop	No server needed. <code>mlflow.set_tracking_uri()</code> defaults to <code>./mlruns</code>
Small team (shared server)	<code>mlflow server --host 0.0.0.0</code> on a shared machine; team sets tracking URI to that IP
Docker container	Mount <code>/mlflow</code> volume; set URI via <code>MLFLOW_TRACKING_URI</code> env variable
CI/CD pipeline	Set <code>MLFLOW_TRACKING_URI</code> as environment variable in your CI system (GitHub Actions, Jenkins)
Databricks	<code>mlflow</code> is pre-installed; tracking is automatic; no server setup needed

► Environment Variables (Alternative to Code Configuration)

```
# Set in your shell or .env file instead of in code
# This keeps your scripts cleaner and environment-agnostic

export MLFLOW_TRACKING_URI='http://localhost:5000'
export MLFLOW_EXPERIMENT_NAME='my-project'

# For S3 artifact store
export AWS_ACCESS_KEY_ID='your-key-id'
export AWS_SECRET_ACCESS_KEY='your-secret-key'
export MLFLOW_S3_ENDPOINT_URL='https://s3.amazonaws.com'
```

► Docker Setup (Production-Ready)

```
# docker-compose.yml for MLflow with PostgreSQL backend
version: '3.8'
services:
  mlflow-db:
    image: postgres:14
    environment:
      POSTGRES_DB: mlflow
      POSTGRES_USER: mlflow
      POSTGRES_PASSWORD: mlflowpass

  mlflow-server:
    image: ghcr.io/mlflow/mlflow:latest
    ports:
      - '5000:5000'
    environment:
      MLFLOW_BACKEND_STORE_URI: postgresql://mlflow:mlflowpass@mlflow-db:5432/mlflow
      MLFLOW_DEFAULT_ARTIFACT_ROOT: /mlartifacts
    depends_on:
      - mlflow-db
    command: mlflow server --host 0.0.0.0 --port 5000
```

► Industry Use Case

Healthcare AI Team — Reproducible Setup Across Environments

A hospital AI team needs their experiments to run identically on: (1) data scientists' local laptops, (2) a GPU training cluster, and (3) a CI/CD pipeline. They solve this by storing `MLFLOW_TRACKING_URI` in each environment's `.env` file. The tracking URI points to a shared PostgreSQL server in all cases. Artifacts go to a HIPAA-compliant S3 bucket with encryption enabled. Every environment logs to the same place with zero code changes — just environment variables differ.

1.4 Logging Parameters

► Definition

Logging parameters means recording the input settings (hyperparameters and configuration choices) of a training run before or during training. Parameters are values that YOU set and that control how the model learns — things like learning rate, number of trees, batch size, optimizer type, or which feature columns you included. Parameters do not change during training; they are fixed inputs.

In MLflow, parameters are logged using `mlflow.log_param()` (single value) or `mlflow.log_params()` (dictionary of multiple values).

► Why It Matters in MLOps

- Without parameter logging, you cannot reproduce a specific run. You may know a model got 92% accuracy, but not what settings produced it.
- Hyperparameter tuning without logging is pointless — you cannot analyse which parameters correlate with better performance.
- Model cards (documentation for deployed models) require exact parameters for audit and compliance.
- AutoML and hyperparameter search tools (Optuna, Ray Tune) integrate with MLflow parameter logging to build searchable, comparable run databases.

► Code & Commands

Basic Parameter Logging

```
import mlflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# Load data
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# --- Define your hyperparameters ---
params = {
    'n_estimators': 100,
    'max_depth': 5,
    'min_samples_split': 10,
    'random_state': 42,
    'class_weight': 'balanced'
}

mlflow.set_experiment('breast-cancer-classifier')

with mlflow.start_run(run_name='rf_baseline_v1'):
    # Log all parameters at once using a dictionary
    mlflow.log_params(params)

    # Or log parameters one by one
    # mlflow.log_param('n_estimators', 100)
    # mlflow.log_param('max_depth', 5)

    # Train the model
    model = RandomForestClassifier(**params)
    model.fit(X_train, y_train)
    print('Training complete!')
```

Logging Non-Hyperparameter Context

```
with mlflow.start_run():
    # Hyperparameters
    mlflow.log_params({
        'learning_rate': 0.001,
        'epochs': 50,
        'batch_size': 32,
        'optimizer': 'adam',
    })

    # Also log important context as parameters
    mlflow.log_param('dataset_version', 'v2.3.1')
    mlflow.log_param('feature_engineering', 'log_transform_skewed')
    mlflow.log_param('train_samples', len(X_train))
    mlflow.log_param('model_type', 'RandomForestClassifier')
    mlflow.log_param('python_version', '3.10.4')
```

► Parameter Rules & Limits

Rule	Detail
Data types	Parameters must be strings, numbers, or booleans. Lists are NOT supported — convert to string: <code>str([1, 2, 3])</code>
Max length	Parameter values have a 500-character limit. Truncate long strings if needed.
Immutable	Parameters logged inside a run cannot be changed after logging. Log them once at the start.
Key naming	Use lowercase_underscore convention. Avoid spaces and special characters.
Max count	MLflow supports up to 100 parameters per run by default (configurable).

► Integration Map

Tool / Library	How Parameters Are Logged
scikit-learn	Manually log <code>model.get_params()</code> dict; or use <code>mlflow.sklearn.autolog()</code> to do it automatically

Optuna (hyperparameter tuning)	Log trial.params dict inside the objective function; each trial becomes one MLflow run
Ray Tune	Use MLflowLoggerCallback — automatically logs all tune config params per trial
Keras / TensorFlow	Log compile() arguments (optimizer, loss, lr) manually or via mlflow.keras.autolog()
Hydra config framework	Log OmegaConf.to_container(cfg) as a flat params dict at run start

► Industry Use Case

E-commerce Recommendation Engine Tuning

A team tunes an XGBoost recommender system with 6 parameters (max_depth, eta, subsample, colsample_bytree, min_child_weight, n_estimators) across 150 runs over 2 days. Because all params are logged with `mlflow.log_params()`, the team uses the MLflow UI to filter for runs where `max_depth < 6` AND `subsample > 0.7` and immediately sees that this combination consistently delivers better offline NDCG@10 scores. This analysis would have taken days manually — MLflow makes it a 10-minute UI query.

1.5 Logging Metrics

► Definition

Logging metrics means recording the output measurements of a training run — the numbers that tell you how well the model is performing. Metrics are values that the model or evaluation process produces, such as accuracy, F1 score, AUC-ROC, loss, mean absolute error, or inference time. Unlike parameters (which are fixed inputs), metrics may change over time during training (e.g., loss per epoch), which is why MLflow supports logging a metric at each timestep (step) to create training curves.

Use `mlflow.log_metric()` for a single value, or `mlflow.log_metrics()` for a dictionary of multiple values. Include the `step` argument to record progression over time.

► Why It Matters in MLOps

- Without metric logging, you cannot compare runs. You might know 'run 47 was the best' but not by how much, on which metric, or at which epoch.
- Training curves (loss per epoch) are essential for diagnosing overfitting, underfitting, or exploding gradients. These are only available if you log metrics with the `step` argument.

- Model monitoring in production compares live metrics against training metrics. Without a logged baseline, you cannot detect drift.
- Business stakeholders need metric history to approve model deployment. A single final accuracy number is insufficient; they want to see convergence curves.

► Code & Commands

Logging Final Evaluation Metrics

```
import mlflow
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score,
precision_score, recall_score

with mlflow.start_run():
    # ... (train your model) ...

    # Get predictions
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    # Log all evaluation metrics at once
    mlflow.log_metrics({
        'accuracy': accuracy_score(y_test, y_pred),
        'f1_score': f1_score(y_test, y_pred, average='weighted'),
        'precision': precision_score(y_test, y_pred),
        'recall': recall_score(y_test, y_pred),
        'auc_roc': roc_auc_score(y_test, y_prob),
    })

    # Log a single metric
    mlflow.log_metric('training_time_seconds', 45.2)
```

Logging Metrics Per Epoch (Training Curves)

```
import mlflow
import numpy as np

# Simulating a neural network training loop
NUM_EPOCHS = 20

with mlflow.start_run():
    for epoch in range(NUM_EPOCHS):
        # Simulate training - replace with real training code
        train_loss = 1.0 / (epoch + 1) + np.random.normal(0, 0.02)
        val_loss = 1.1 / (epoch + 1) + np.random.normal(0, 0.03)
```

```

train_acc = 1 - train_loss * 0.8
val_acc = 1 - val_loss * 0.8

# Log metrics at each epoch using 'step'
# 'step' is the x-axis in the MLflow UI chart
mlflow.log_metric('train_loss', train_loss, step=epoch)
mlflow.log_metric('val_loss', val_loss, step=epoch)
mlflow.log_metric('train_acc', train_acc, step=epoch)
mlflow.log_metric('val_acc', val_acc, step=epoch)

print(f'Epoch {epoch+1}/{NUM_EPOCHS} - val_loss: {val_loss:.4f}')

print('Training complete. Metrics logged to MLflow!')

```

► Metrics vs. Parameters — Key Difference

Concept	Parameters
What it is	Inputs YOU set before training
Changes during training?	No — fixed before training starts
Examples	learning_rate, n_estimators, batch_size
MLflow function	mlflow.log_param() / log_params()
Supports step?	No

► Industry Use Case

NLP Team Tracking BERT Fine-Tuning

A team fine-tunes BERT on customer support tickets to classify intent. Training takes 3 hours per run on a GPU cluster. By logging `val_loss` and `val_f1` per epoch, the team discovers in the MLflow UI that after epoch 4, `val_loss` starts rising (overfitting) while `train_loss` continues to fall. They add early stopping at epoch 4 for future runs, reducing GPU compute cost by 60% — saving roughly \$800/month in cloud costs. This insight was only possible because epoch-level metrics were captured with the step parameter.

1.6 Logging Artifacts

► Definition

Logging artifacts means saving any file produced by your training run to MLflow's artifact store, so it is permanently linked to that specific run. Artifacts are binary or text files — not simple numbers. Common artifacts include: the trained model file (.pkl, .h5, .pt), evaluation plots (confusion matrix, ROC curve as .png), feature importance charts, the data preprocessing pipeline, tokenizers, configuration files, or even the actual dataset used for training.

Use `mlflow.log_artifact()` to upload a single file, `mlflow.log_artifacts()` to upload an entire folder, or model-specific functions like `mlflow.sklearn.log_model()` to log a complete model with schema and metadata.

► Why It Matters in MLOps

- Without artifact logging, 'the best model' has no associated file. You know which parameters worked, but the actual model binary might be overwritten or lost.
- Deployment pipelines need a consistent model file path. MLflow artifacts give every model a permanent, retrievable URI like `runs:/RUN_ID/model`.
- Debugging production failures requires the exact model file and preprocessing pipeline from that training run. Artifacts make this possible.
- Reproducibility audits (required in finance, healthcare, insurance) demand that the exact model artifact used for predictions is preserved and retrievable.

► Code & Commands

Logging a Trained Model File

```
import mlflow
import mlflow.sklearn
import pickle
from sklearn.ensemble import GradientBoostingClassifier

with mlflow.start_run():
    model = GradientBoostingClassifier(n_estimators=200, learning_rate=0.05)
    model.fit(X_train, y_train)

    # Method 1: Save model file manually, then log it
    with open('model.pkl', 'wb') as f:
        pickle.dump(model, f)
    mlflow.log_artifact('model.pkl') # uploads to artifact store

    # Method 2 (RECOMMENDED): Use MLflow's native model logging
    # This adds schema, signature, and makes the model deployable
    mlflow.sklearn.log_model(
        sk_model=model,
        artifact_path='model', # subfolder in artifact store
        registered_model_name='fraud-detector' # register in Model Registry
    )
```

Logging Plots and Reports

```
import mlflow
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
import os

with mlflow.start_run():
    # --- Confusion Matrix ---
    fig, ax = plt.subplots(figsize=(6, 5))
    ConfusionMatrixDisplay.from_predictions(y_test, y_pred, ax=ax)
    plt.title('Confusion Matrix - Test Set')
    plt.tight_layout()
    plt.savefig('confusion_matrix.png', dpi=150)
    mlflow.log_artifact('confusion_matrix.png', artifact_path='plots')
    plt.close()

    # --- ROC Curve ---
    fig, ax = plt.subplots(figsize=(6, 5))
    RocCurveDisplay.from_predictions(y_test, y_prob, ax=ax)
    plt.title('ROC Curve - Test Set')
    plt.tight_layout()
    plt.savefig('roc_curve.png', dpi=150)
    mlflow.log_artifact('roc_curve.png', artifact_path='plots')
    plt.close()

    # --- Log an entire reports folder ---
    os.makedirs('reports', exist_ok=True)
    # ... (write files to reports/) ...
    mlflow.log_artifacts('reports/', artifact_path='reports')
```

Logging the Preprocessing Pipeline

```
import mlflow
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

# Build preprocessing pipeline
numeric_features = ['age', 'income', 'credit_score']
categorical_features = ['employment_type', 'region']

preprocessor = ColumnTransformer([
```

```

    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
])

# Build full pipeline: preprocessing + model
full_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', GradientBoostingClassifier())
])

with mlflow.start_run():
    full_pipeline.fit(X_train, y_train)

# Log the ENTIRE pipeline as one model artifact
# This ensures preprocessing + model are always together
mlflow.sklearn.log_model(full_pipeline, 'full_pipeline')

```

► Artifact Types Quick Reference

Artifact Type	Typical Files
Trained model	model.pkl, model.h5, model.pt
Preprocessing pipeline	preprocessor.pkl, scaler.pkl
Evaluation plots	confusion_matrix.png, roc.png
Feature importance	feature_importance.csv, feat_imp.png
Config files	config.yaml, hyperparams.json
Entire folder	reports/, outputs/

► Retrieving Artifacts Programmatically

```

import mlflow

# Download an artifact from a specific run
run_id = 'abc123def456' # get this from MLflow UI

# Option 1: Load a logged sklearn model directly
model = mlflow.sklearn.load_model(f'runs:{run_id}/full_pipeline')
predictions = model.predict(X_new)

# Option 2: Download artifact file to local disk
local_path = mlflow.artifacts.download_artifacts(
    f'runs:{run_id}/plots/confusion_matrix.png'
)

```

```
)
print(f'Downloaded to: {local_path}')
```

► Industry Use Case

🔒 Insurance Company — Audit-Ready Artifact Storage

An insurance company deploys a claims fraud detection model. Regulators require that for any claim decision, the company can produce: (1) the exact model used, (2) the preprocessing pipeline, (3) training data statistics, and (4) validation plots. Using MLflow artifacts, every model version has all four files permanently linked to a `run_id`. When auditors request the model from 18 months ago, the team retrieves `run_id=ab42c9` and downloads all artifacts in 30 seconds — a process that previously required 2 days of archaeology through shared drives.

1.7 Auto-Logging with `mlflow.autolog()`

► Definition

Auto-logging is MLflow's 'zero-code' tracking feature. When you call `mlflow.autolog()` (or a library-specific variant like `mlflow.sklearn.autolog()`), MLflow automatically intercepts calls to supported ML libraries and logs parameters, metrics, and model artifacts without you writing any `mlflow.log_*` calls manually.

It works through monkey-patching — MLflow wraps the training functions of popular libraries (`fit()`, `train()`, etc.) with its own code that extracts and logs everything automatically. You simply add one line and get complete tracking for free.

► Why It Matters in MLOps

- Manual logging requires 20–30 lines of boilerplate per experiment. Teams under deadline pressure skip it — leading to untracked experiments.
- New team members forget to add `log_param()` calls, creating inconsistent tracking across the team.
- Auto-logging enforces a consistent logging standard across all runs with zero discipline required.
- It captures metrics that you might not think to log manually — for example, feature importances, training time, and per-class metrics.
- In fast-iteration phases (early EDA, quick prototyping), auto-logging lets you track without slowing down.

► Code & Commands

Enabling Global Auto-Logging

```
import mlflow

# Enable auto-logging for ALL supported libraries at once
# Put this ONCE at the top of your script or notebook
mlflow.autolog()

# That's it! Now just train your model normally.

from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# MLflow automatically starts a run and logs everything
model = LogisticRegression(C=0.5, max_iter=500, solver='lbfgs')
model.fit(X_train, y_train)

# MLflow auto-logged:
# Parameters: C=0.5, max_iter=500, solver='lbfgs', multi_class='auto', etc.
# Metrics: training_score (accuracy on train set)
# Artifacts: model.pkl + MLmodel metadata file
```

Library-Specific Auto-Logging

```
# Enable only for specific libraries (more control)

# --- scikit-learn ---
mlflow.sklearn.autolog(
    log_input_examples=True, # logs sample input for model signature
    log_model_signatures=True, # logs input/output schema
    log_models=True, # logs the model artifact
    log_post_training_metrics=True, # runs evaluation after fit()
)

# --- XGBoost ---
mlflow.xgboost.autolog(
    importance_types=['gain', 'weight'] # log feature importances
)

# --- LightGBM ---
mlflow.lightgbm.autolog()
```

```
# --- PyTorch Lightning ---
mlflow.pytorch.autolog(log_every_n_epoch=1)

# --- TensorFlow / Keras ---
mlflow.tensorflow.autolog(
    every_n_iter=1, # log metrics every epoch
    log_models=True,
    log_datasets=False
)
```

Auto-Logging with Manual Overrides

```
import mlflow

mlflow.autolog()

with mlflow.start_run(run_name='experiment_with_custom_tags'):
    # Let auto-logging handle params, metrics, and model
    model = RandomForestClassifier(n_estimators=200, max_depth=8)
    model.fit(X_train, y_train)

    # ADD your own custom metrics on top of auto-logged ones
    mlflow.log_metric('business_metric_revenue_impact', 12500.0)

    # ADD custom artifacts
    mlflow.log_artifact('my_custom_report.html', 'reports')

    # ADD tags to categorise the run
    mlflow.set_tag('team', 'fraud-detection')
    mlflow.set_tag('data_version', 'Q4-2024')
    mlflow.set_tag('reviewer', 'alice@company.com')
```

► What Gets Auto-Logged Per Library

Library	Auto-Logged Parameters
scikit-learn	All model.__init__ params
XGBoost	All booster params
LightGBM	All training params
Keras/TF	optimizer, loss, lr, batch_size

► Disabling Auto-Logging Selectively

```
import mlflow

# Enable globally
mlflow.autolog()

# Disable for a specific block of code
with mlflow.start_run():
    with mlflow.utils.autologging_utils.disable_autologging():
        # This training run will NOT be auto-logged
        model = LogisticRegression().fit(X_train, y_train)

# Or disable entirely
mlflow.autolog(disable=True)

# Turn off specific libraries only
mlflow.sklearn.autolog(disable=True)
mlflow.xgboost.autolog() # still active
```

► Integration Map

Framework	Auto-Log Call
scikit-learn	mlflow.sklearn.autolog()
XGBoost	mlflow.xgboost.autolog()
LightGBM	mlflow.lightgbm.autolog()
TensorFlow/Keras	mlflow.tensorflow.autolog()
PyTorch Lightning	mlflow.pytorch.autolog()
Spark MLlib	mlflow.spark.autolog()

► Industry Use Case

⚡ Fast Startup — 10x Experiment Velocity with Auto-Logging

A 3-person data science team at a fintech startup needs to prototype 5 different model families (LogReg, SVM, GBM, XGBoost, LightGBM) in a 2-day sprint before a board presentation. They add `mlflow.autolog()` as the first line of each notebook. By end of day 2, they have 47 fully-tracked runs

across all model types with zero manual logging code. The UI comparison shows LightGBM with specific defaults beats XGBoost by 2.3% AUC-ROC. Without auto-logging, they would have spent half the sprint writing `log_param()` calls instead of experimenting.

1.8 Comparing Runs in the MLflow UI

► Definition

The MLflow UI is a web-based dashboard that displays all your logged experiments in an interactive table and chart format. Comparing runs means using the UI to place multiple training runs side by side, filter them by metric thresholds, sort them by performance, and visualise how parameters influence metrics through charts and parallel coordinate plots.

The UI is automatically served by the MLflow tracking server at `http://localhost:5000` (or your remote server URL). No additional setup is needed — it is built into MLflow.

► Why It Matters in MLOps

- Without a visual comparison tool, selecting the best model from 50+ runs requires manually reading log files — error-prone and time-consuming.
- Parallel coordinate plots reveal non-obvious relationships between hyperparameters and performance that tables cannot show.
- The UI enables non-technical stakeholders (managers, product owners) to see experiment progress without needing to read code.
- The Model Registry integration (reachable from the UI) allows promoting the best run to staging or production with a single click.
- Searching and filtering runs in the UI replaces complex pandas queries, making exploration accessible to junior team members.

► Step-by-Step Workflow — Using the MLflow UI

16. Launch the UI: run `mlflow ui` in your terminal (local) or navigate to your server URL. The default port is 5000.
17. Select your Experiment from the left sidebar (e.g., 'credit-risk-model'). You will see a table of all runs.
18. Search/Filter runs using the search bar. Examples: `metrics.accuracy > 0.90`, `params.n_estimators = '200'`, `tags.team = 'fraud'`.
19. Sort runs by clicking any column header (click 'accuracy' to sort by accuracy descending).
20. Select multiple runs using the checkboxes on the left, then click 'Compare' to open the comparison view.

21. In the comparison view, toggle between Table View (side-by-side params and metrics) and Chart View (line charts, scatter plots, parallel coordinates).
22. Click any run name to open its detail page — see all params, metrics, and a file browser for artifacts.
23. From the detail page, click 'Register Model' to promote the best run to the Model Registry.

► UI Features Deep Dive

UI Feature	What It Does & How to Use It
Runs Table	Main dashboard view. Shows run name, date, duration, params, and metrics as columns. Click column headers to sort.
Search Bar	Supports MLflow Search DSL. Syntax: <code>metrics.f1_score > 0.85 AND params.max_depth = '6'</code> . Use AND/OR to combine.
Compare View	Select 2+ runs via checkboxes → click Compare. Shows a diff table highlighting where runs differ in params and metrics.
Parallel Coordinates Plot	Visual line plot where each vertical axis is one param or metric. Lines that converge on good metrics reveal effective param combinations.
Scatter Plot	Plot any metric on X-axis vs. any metric on Y-axis. Useful for accuracy vs. training_time tradeoff analysis.
Metric Charts	For runs with step-based metrics, shows time series charts (loss per epoch). Only visible when step > 0 was used during logging.
Artifact Browser	File tree view of all logged artifacts for a run. Click to preview images; download buttons for model files.
Run Detail Page	Full view of a single run — all params, all metrics, source code info, git commit hash, and artifact browser.

► MLflow Search Query Language — Examples

```
# In the Search Runs bar at the top of the Experiments table:

# Find runs with accuracy above 90%
metrics.accuracy > 0.90

# Find runs using a specific parameter value
params.n_estimators = '200'

# Combine multiple conditions
```

```
metrics.accuracy > 0.88 AND params.max_depth = '5'

# Find runs by tag
tags.team = 'fraud-detection'

# Filter by run status
attributes.status = 'FINISHED'

# Using the Python client to search runs programmatically
import mlflow

results = mlflow.search_runs(
    experiment_names=['credit-risk-model'],
    filter_string='metrics.accuracy > 0.90 AND params.n_estimators = "200"',
    order_by=['metrics.accuracy DESC'],
    max_results=10
)
print(results[['run_id', 'metrics.accuracy', 'params.n_estimators']].head())
```

► Programmatic Run Comparison (Python API)

```
import mlflow
import pandas as pd

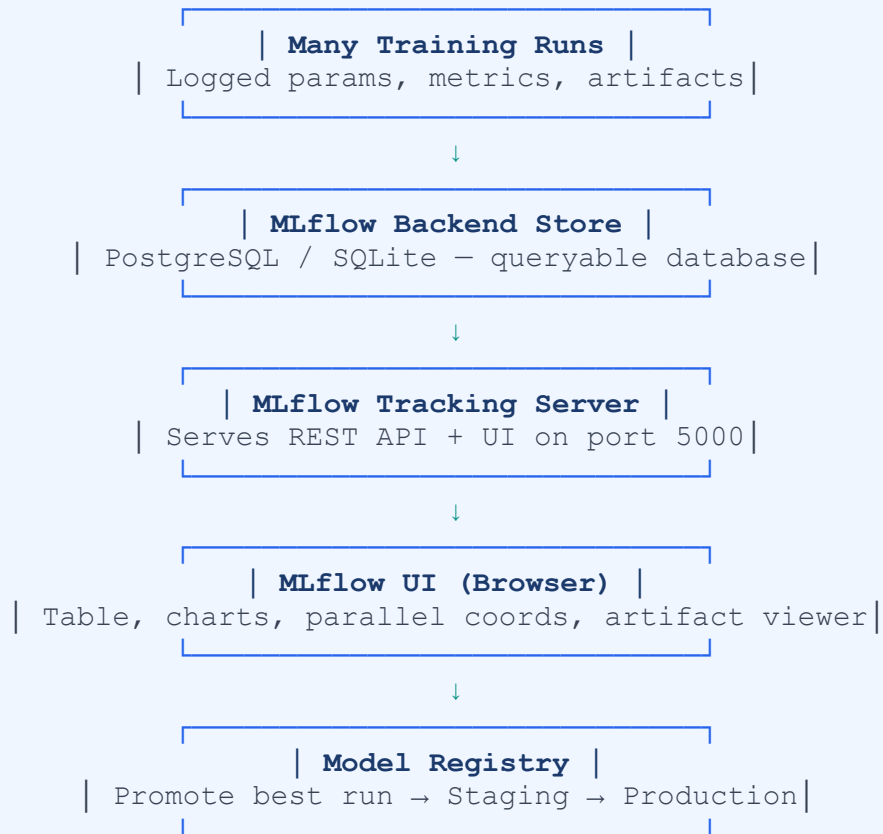
# Search all finished runs in an experiment
df = mlflow.search_runs(
    experiment_names=['breast-cancer-classifier'],
    filter_string='attributes.status = "FINISHED"',
    order_by=['metrics.auc_roc DESC']
)

# Show top 5 runs by AUC-ROC
top5 = df[['run_id', 'metrics.auc_roc', 'metrics.f1_score',
    'params.n_estimators', 'params.max_depth']].head(5)
print(top5.to_string(index=False))

# Get the best run ID
best_run_id = df.loc[df['metrics.auc_roc'].idxmax(), 'run_id']
print(f'Best run: {best_run_id}')

# Load the best model
best_model = mlflow.sklearn.load_model(f'runs://{best_run_id}/model')
```

► System Architecture — UI in Context



► Integration Map

Tool	How It Integrates with MLflow UI
Jupyter Notebooks	Run mlflow ui in terminal; open browser tab alongside notebook for live tracking
VS Code	Use the MLflow extension for VS Code to view runs inside the editor
Databricks	MLflow UI is natively embedded in the Databricks workspace — no separate browser needed
GitHub Actions (CI/CD)	Use mlflow.search_runs() in CI to auto-select and register the best model after a test suite
Grafana	Export metric history from MLflow backend store into Grafana for production dashboards
Slack / Teams	Use Python API to auto-post best run results to team channels after overnight training jobs

► Industry Use Case

Weekly Model Review Meeting Made Effortless

A data science team holds weekly model review meetings with the Head of Product. Previously, data scientists spent 3 hours preparing comparison spreadsheets before each meeting. After adopting MLflow, they simply open the MLflow UI on the projector, filter for runs from the past week (`start_time > DATE`), sort by `f1_score`, and use the parallel coordinates plot to show how `learning_rate` and `max_depth` interact with performance. The Product Head can now see and understand experiment progress in real time. Meeting prep time: 0 hours. Decision quality: significantly higher because they see all runs, not a curated subset.

Chapter 1 — Quick Summary Chart

Use this table as a rapid reference for the entire chapter. Review it before interviews, exams, or deploying your first MLflow-tracked model.

Subtopic	Core Concept	Key MLflow API	Common Pitfall	One-Line Test
1.1 — What is MLflow	Open-source ML lifecycle platform; experiment tracking is its core feature	<code>mlflow.start_run()</code>	Confusing MLflow with model serving tools like TorchServe	Can you name 3 things that break without experiment tracking?
1.2 — Architecture	4 components: Tracking Server, Backend Store, Artifact Store, UI	<code>mlflow.set_tracking_uri()</code>	Running local-only setup in a team context — no one shares results	Where are metrics stored vs. where are model files stored?
1.3 — Setup	<code>pip install mlflow</code> ; start server; <code>set_tracking_uri</code> ; <code>set_experiment</code>	<code>mlflow.set_experiment()</code>	Forgetting to set tracking URI — everything goes to local <code>./mlruns</code> silently	What command starts the MLflow server with a PostgreSQL backend?
1.4 — Log Parameters	Record fixed inputs (hyperparams) before/during training with <code>log_params()</code>	<code>mlflow.log_params(dict)</code>	Logging a list as a parameter — must convert to string first	What is the difference between a parameter and a metric?
1.5 — Log Metrics	Record output scores; use <code>step=</code> for time-series charts (loss per epoch)	<code>mlflow.log_metric(name, val, step=)</code>	Forgetting <code>step=</code> — losing training curve data, only seeing final values	How do you log loss per epoch so it appears as a chart in the UI?

1.6 — Log Artifacts	Save any file (model, plot, pipeline) permanently linked to a run	<code>mlflow.sklearn.log_model()</code>	Logging the model only — not the preprocessing pipeline (breaks deployment)	What URI pattern retrieves a model from a specific run?
1.7 — Auto-Logging	One-line tracking for sklearn, XGBoost, LightGBM, Keras, PyTorch	<code>mlflow.autolog()</code>	Using autolog without reviewing what it logs — may miss business metrics	How do you add custom metrics on top of auto-logged ones?
1.8 — MLflow UI	Web dashboard for filtering, comparing, and visualising runs	<code>mlflow.search_runs()</code>	Not using tags — runs become impossible to filter by team/project	Write a search query to find runs with accuracy > 0.90